

Digit Detection with DETR and ResNet as backbone

VRDL Homework 2

Hung-I Yang 112550097

April 18, 2026

1 Introduction

1.1 Task description

This homework is a *digit detection* task using a DETR with a ResNet backbone. The dataset is provided in COCO format, where each object is annotated by class label and bounding box $([x_{\min}, y_{\min}, w, h])$. The official split contains approximately 30,062 training images, 3,340 validation images, and 13,068 hidden-test images. The goal is to predict all digit instances in each image and use mAP to evaluate the performance.

1.2 Core idea

- **Replace vanilla DETR with DINO.** DETR’s randomly initialized queries force the Hungarian matcher to simultaneously solve query-to-object assignment and box regression from scratch, leading to notoriously slow convergence (~ 500 epochs [5]). DINO [5] resolves this by (i) *mixed query selection*—seeding positional queries from top encoder outputs rather than random vectors; (ii) *contrastive denoising training (CDN)*—injecting noised ground-truth boxes as explicit regression targets, decoupled from bipartite matching; and (iii) *look-forward-twice*—propagating gradients from later decoder layers back to earlier ones for consistent iterative refinement.
- **Shallower transformer depth.** The full DETR/DINO default of 6 encoder/decoder layers is designed for COCO’s 80-class density. Our dataset has only 10 classes and ~ 2.5 boxes per image, so I reduce to 3 layers each—cutting compute and overfitting risk while preserving the capacity needed for digit detection.
- **Higher input resolution.** ResNet-50 downsamples by $32\times$, leaving as few as 3×5 spatial tokens at 100px input. I upscale to a 320×640 short/long-side cap so the multi-scale feature pyramid has enough tokens for deformable attention to sample digit strokes accurately.
- **COCO-style augmentation.** With $\sim 30,000$ training images, augmentations (color jitter, blur, perspective, crop) act as a regularizer against background memorization and improve robustness under the distribution shift between training and test images.

Details and ablation results for each design choice are discussed in Sections 2.1 and 3.1.1. Training and inference code are available at <https://github.com/harry0816web/NYCU-VRDL/tree/main/hw2>.

2 Method

2.1 Data preprocessing and augmentation

Annotations are converted from COCO bbox to axis-aligned $[x_0, y_0, x_1, y_1]$, clamped to image bounds, with invalid boxes removed. Then I apply the following augmentations:

- **Pixel-level augmentations.**
 - ColorJitter with $p=0.8$.
 - Gaussian blur with $p=0.5$ and `augment_blur_sigma`=[0.1,2.0].
 - ISO noise with $p=0.2$.
- **Geometric augmentations.**
 - RandomPerspective with $p=0.5$ and `scale`=0.1.
- **Spatial sampling.**
 - RandomCrop with $p=0.5$ and `scale`=[0.6,1.0].
- **Resize strategy.**
 - Base resize: `train_short_edge`=320 and `train_max_size`=640. First resize to the short edge, then resize long edge to 640 if it’s larger than 640.
 - Additional random scale: `train_random_scale_min/max`=[0.9,1.0].
- **Final preprocessing and box filtering.**
 - Apply ImageNet normalization and convert boxes to normalized center–width–height format.
 - Remove boxes with visible area ratio below `augment_min_visibility`=0.5 after geometric transforms.

Validation preprocessing is deterministic:

- Resize with `val_short_edge`=320 and `val_max_size`=640.
- Apply normalization only (no stochastic augmentation).

2.2 Model architecture

2.2.1 Backbone (ResNet-50 [2])

Vanilla DETR [1] feeds only the last ResNet stage (C5, stride 32) into the transformer, discarding the finer spatial detail in earlier stages that is critical for small-object localization. Following Deformable DETR [6], DINO instead constructs a **multi-scale feature pyramid**: intermediate feature maps at multiple backbone stages (strides 8, 16, 32) are each projected to the model dimension and concatenated into a single token sequence before being passed to the encoder. An additional coarser scale is synthesized from the last backbone output to cover large receptive fields. Each scale is distinguished by a learnable level embedding, and the encoder uses deformable attention [6] to attend efficiently across all scales simultaneously, enabling the model to detect objects at varying sizes without the quadratic cost of global attention.

2.2.2 DINO: improvements over vanilla DETR

Vanilla DETR [1] uses a fixed set of object queries decoded by a transformer encoder–decoder, with predictions matched to ground-truth boxes via the Hungarian algorithm. DINO [5] preserves this set-prediction framework but introduces five targeted improvements to address convergence instability and detection quality.

1. **Mixed query selection.** Vanilla DETR initializes positional queries from scratch, yielding no spatial prior in early training. DINO selects the top- K encoder output locations to seed *positional* queries as 4D reference boxes while keeping *content* queries learnable, so the decoder starts from salient regions rather than random anchors.
2. **Contrastive denoising training (CDN).** Bipartite matching is unstable when predictions are random at initialization. DINO injects noised ground-truth boxes as extra decoder queries—small-noise copies are pushed to reconstruct the GT box, while large-noise copies are treated as negatives. This provides a dense regression signal decoupled from the fragile matching process.
3. **Look-forward-twice and iterative box refinement.** Rather than predicting boxes independently at each decoder layer, DINO refines reference boxes layer-by-layer with predicted offsets, and lets gradients from deeper layers flow back to shallower ones, encouraging consistent incremental localization improvement.
4. **Multi-scale features and deformable attention.** As described in Section 2.2.1, DINO replaces the single C5 feature map and costly global attention of vanilla DETR with a multi-scale pyramid and deformable attention [6], improving both small-object recall and computational efficiency.
5. **Sigmoid focal classification.** DETR’s softmax over all classes with a dedicated “no-object” logit struggles under heavy foreground–background imbalance. DINO uses per-class sigmoid predictions with focal loss [3], treating background as all classes scoring near zero.

Loss function. DINO’s total training loss combines a focal classification loss, an ℓ_1 bounding-box regression loss, and a generalized IoU (GIoU) loss for the matched queries, plus an additional reconstruction loss from the CDN branch. Following DETR [1], the same set of losses is also applied as auxiliary losses at every intermediate decoder layer.

2.3 Hyperparameters and training strategy

Starting from the default settings of the original DINO implementation, training curves are monitored with Weights & Biases (W&B) and hyperparameters are iteratively refined based on the results of each experiment. Table 1 summarizes the final training configuration.

3 Results

3.1 Training & Validation Curves

3.1.1 DETR vs. DINO

As motivated in Section 2.2.1, I adopt DINO over vanilla DETR for its mixed query selection, contrastive denoising training, and look-forward-twice refinement. To empirically verify these advantages on our dataset, I train both architectures under an identical compact configuration: encoder/decoder depth = 3, `hidden_dim` = 128, `dim_feedforward` = 512, and training short-side randomly resized to 100–200 pixels.

Table 1: Final DINO training configuration.

Field	Value
lr / lr_backbone / lr_linear_proj_mult	10^{-4} / 10^{-5} / 0.1
batch_size	8
epochs	20
weight_decay	10^{-4}
dropout	0.2
num_queries / num_select	30 / 30
dn_number	10
Transformer depth	enc_layers=3, dec_layers=3
hidden_dim / nheads / dim_feedforward	256 / 4 / 512
LR scheduler	Cosine annealing, lr_eta_min=1e-6
Matcher costs (set_cost_class, bbox, giou)	2.0, 5.0, 2.0

As shown in Figure 1, DINO reaches a competitive mAP within the first few epochs while DETR’s validation mAP rises much more slowly, a gain attributed primarily to mixed query selection and contrastive denoising training [5]. Moreover, as reported in Table 2, the test score of DINO substantially outperforms the DETR baseline.



Figure 1: DETR vs. DINO: training loss and validation loss (top row); validation mAP@50, mAP@75, and mAP@[50:95] (bottom row).

Table 2: Test mAP@[0.50:0.95] for DETR and DINO baselines (compact architecture).

Model	mAP@[.50:.95]
DETR baseline	0.28
DINO baseline	0.36

3.1.2 Effect of Data Augmentation and Hidden Dimension

We next evaluate the effect of the data augmentation pipeline described in Section 2.1. Applying full augmentation to the DINO baseline (`hidden_dim` = 128) produces a counter-intuitive outcome: both the validation bounding-box regression loss (`loss_bbox`) and the generalized-IoU loss (`loss_giou`) fail to decrease—they rise or plateau—and the training loss no longer descends as cleanly as in the unaugmented run.

I hypothesize that the compact model lacks the representational capacity to absorb the increased input variance introduced by heavy augmentation. To address this, I increase `hidden_dim` to 256, keeping all other hyperparameters fixed.

Figure 2 shows two curves side by side for both augmented variants. However, as can be seen in Table 3, test score didn’t improve as expected.



Figure 2: Augmentation and hidden-dimension experiments: training loss and validation loss (top row); validation mAP@50, mAP@75, and mAP@[50:95] (bottom row).

Table 3: Test mAP@[0.50:0.95]: effect of augmentation and larger hidden dimension.

Setting	mAP@[.50:.95]
DINO + aug (hidden_dim=128)	0.35
DINO + aug (hidden_dim=256)	0.35

3.1.3 Input Resolution Upscaling (320×640)

The 100–200 pixel short-side used in prior experiments produces very few spatial tokens after backbone downsampling, limiting the multi-scale context available to the encoder. We therefore upscale the input to 320×640 (`train_short_edge=320`, `train_max_size=640`) to allow the encoder to extract richer features. As shown in Figure 3, the higher-resolution model converges to a strong result more quickly. What’s more, Table 4 shows that it achieves current best test score.

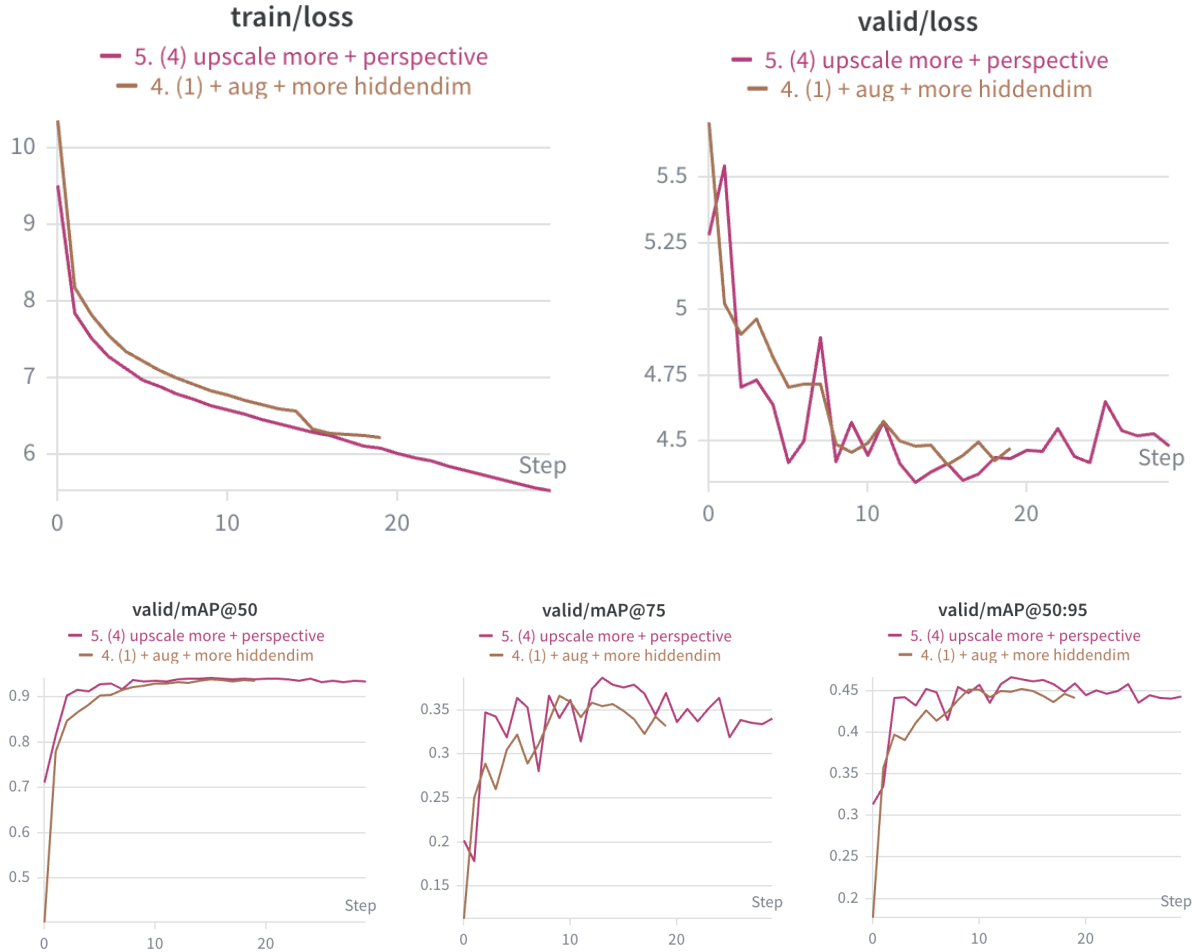


Figure 3: Input resolution upscaling to 320×640: training loss and validation loss (top row); validation mAP@50, mAP@75, and mAP@[50:95] (bottom row).

Table 4: Test mAP@[0.50:0.95]: effect of input resolution upscaling.

Setting	mAP@[.50:.95]
Previous result (100–200 short-side)	0.35
Previous result + upscaled (320×640)	0.37

3.1.4 Denoising Query Ratio (dn_number Adjustment)

Across all augmentation and resolution settings, I observe a consistent anomaly: `loss_giou` does not decrease monotonically—it rises during early training before eventually declining. I trace this to a severe imbalance in the decoder’s query budget. With the initial setting of `num_queries` = 50 and `dn_number` = 50, the CDN groups consume

$$\text{dn_number} \times 2 \times \bar{N}_{\text{box}} \approx 50 \times 2 \times 2.5 = 250$$

query slots per forward pass (estimated using the average of $\bar{N}_{\text{box}} \approx 2.5$ GT boxes per image), leaving the genuine matching queries to represent only $50/(50 + 250) \approx 16.7\%$ of all decoder tokens. At this ratio, the bipartite-matching branch is severely starved of regression feedback—especially early in training when CDN supervision dominates and the matching signal has the most influence on localization convergence.

To restore a healthier balance, I reduce the denoising budget to `dn_number` = 10 and set `num_queries` = 30, so that CDN queries occupy only $10 \times 2 \times 2.5 = 50$ slots and the matching queries represent $30/(30 + 50) \approx 37.5\%$ of all decoder tokens. This gives the bipartite-matching branch substantially more localization supervision at every iteration, which I expect to stabilize `loss_giou` and improve final accuracy.

Figure 4 shows the corresponding optimization curves after reducing `dn_number`. Table 5 reports the test score improvement.



Figure 4: Effect of denoising-query ratio tuning (`dn_number` adjustment): training loss and validation loss (top row); validation mAP@50, mAP@75, and mAP@[50:95] (bottom row).

Table 5: Test mAP@[0.50:0.95]: effect of denoising-query ratio tuning.

Setting	Test score
Previous best (before <code>dn_number</code> adjustment)	0.37
Reduced <code>dn_number</code> (denoising-query ratio tuned)	0.39

4 Additional Experiments

4.1 Experiment: Ensemble Learning — Bagging with Multi-Seed Training and WBF

Hypothesis & How this may work. By training multiple models that are *identical in architecture and hyperparameters* but differ only in their random seed, and then fusing their predictions, I expect the ensemble to reduce per-model variance and produce more stable, better-calibrated bounding boxes—an approach analogous to bagging in classical ensemble learning. I select **Weighted Boxes Fusion (WBF)** [4] as the fusion strategy over standard NMS because

WBF retains and averages all overlapping predictions across models rather than discarding lower-scoring boxes, making fuller use of each model’s localization information.

Mechanism. Three DINO models are trained independently using seeds 0, 42, and 67, sharing the same config. Each seed introduces distinct randomness into weight initialization, data augmentation sampling, and CDN label/box noise. At inference time, each model independently produces `top_num_select` predictions per image. The three sets of predictions are then fused per image via WBF:

1. All predicted boxes across models are sorted by confidence score.
2. Each box is matched to an existing cluster if $\text{IoU} > \text{iou_thr}$ ($= 0.55$); otherwise a new cluster is created.
3. The fused box coordinate is the weighted average of all boxes in the cluster; the fused score is the sum of cluster scores divided by the total number of models.

Boxes with score below `skip_box_thr` ($= 0.0001$) are excluded before fusion. The fused normalized coordinates are then converted to absolute-pixel xywh format for COCO submission.

Results. Table 6 reports the test mAP@[0.50:0.95] before and after ensembling.

Table 6: Test mAP@[0.50:0.95] : single model vs. multi-seed WBF ensemble.

Setting	mAP@[.50:.95]
Single model (seed 42)	0.39
Multi-seed WBF ensemble (seeds 0, 42, 67)	0.39

The ensemble yields no improvement over the single-model baseline. The primary benefit of bagging is variance reduction, but this gain diminishes when the base models are already heavily regularized. In our setting, data augmentation (color jitter, Gaussian blur, ISO noise, random perspective, random crop), weight decay, and dropout had already substantially constrained each model’s variance before ensembling. With per-seed variance already low, models trained with different seeds converge to similar solutions, leaving little complementary information for WBF to exploit.

References

- [1] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*, 2020.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [3] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2017.
- [4] Roman Solovyev, Weicheng Wang, and Tatiana Gabruseva. Weighted boxes fusion: Ensembling boxes from different object detection models. *Image and Vision Computing*, 107: 104117, 2021.

- [5] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In *International Conference on Learning Representations (ICLR)*, 2023.
- [6] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable DETR: Deformable transformers for end-to-end object detection. In *International Conference on Learning Representations (ICLR)*, 2021.